# Lab 5

DSA

February 6, 2026

## Exercise

Given an array of $n$ integers. Apply **Quicksort** to the array using the following rules:

- A pivot element is chosen randomly from the current subarray.
- The subarray is partitioned using the standard partitioning method discussed in class.
- After partitioning, elements smaller than the pivot appear in the left subarray and elements greater than the pivot appear in the right subarray.
- The same procedure is recursively applied to the left and right subarrays.
- The recursion stops when the subarray size becomes less than or equal to one.

## Expected Output

**Input array:**

`10 3 7 4 8 2 9 1 6 5`

**Sorted array:**

`1 2 3 4 5 6 7 8 9 10`

### Random Pivot Selection

```
int randomPivot(int low, int high) {
    return low + rand() % (high - low + 1);
}
```

**Explanation:** The above function `randomPivot` selects an index uniformly at random from the range `[low, high]`. Choosing a random pivot reduces the likelihood of encountering the worst-case behavior of Quicksort (such as when the input array is already sorted or reverse sorted) and helps achieve an expected time complexity of $O(n \log n)$. You may use the above method for selecting a random pivot element.

### Partitioning Approach

1. Choose a pivot element from the subarray (pivot selection must be random).

2. Move the chosen pivot to the beginning (0th index) of the subarray.

3. Initialize an index $i$ to one position before the start of the subarray.

4. Traverse the subarray from left to right (excluding the pivot):

   - If the current element is less than or equal to the pivot, increment $i$ and swap the current element with the element at index $i$.

5. After traversal, swap the pivot element with the element at index $i + 1$.

6. The pivot is now placed at its correct sorted position.

## Task

Write a C program to implement Quicksort using a random pivot and the partitioning scheme discussed in class. [CO1, CO2, CO3]

## Assessment Rubric

| **Assessment Criteria** | **Marks** |
|---|---|
| Correct Random Pivot Selection | 2 |
| Correct Partitioning | 5 |
| Correct Recursive Calls | 2 |
| Correct Sorted Output | 1 |
| **Total** | **10** |